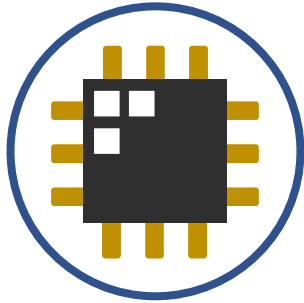




EEE 113:

Introduction to Electrical and Electronics Engineering Systems

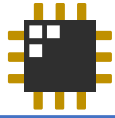
Week 1 –Number systems

1

Welcome to Week 1! Every week we'll be releasing two slide sets, one for each meeting.

We are now in the computing section of EEE 113.

This is part 1.



Introduction

- There are several abstraction layers in a computer system
 - EEE 111 is on the highest level (high-level programming using Python, C, etc)
 - EEE 113 (this course) briefly touches the lower levels of hierarchy
 - How a computer *really* does what it does.
 - Looking forward, we will be *very briefly* touching the topics covered in EEE 143 and EEE 153... **in 5 weeks!**

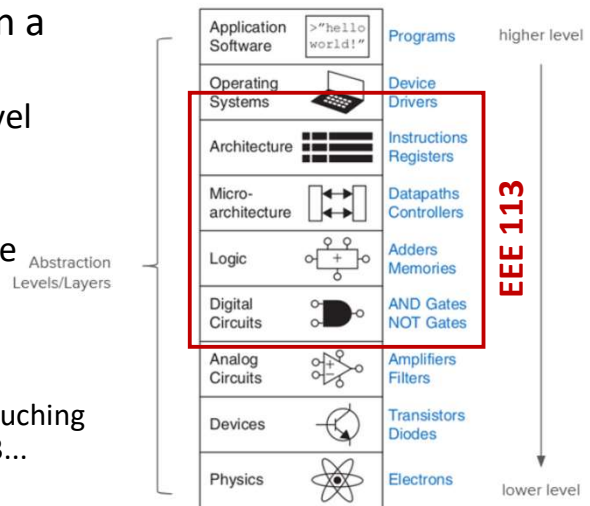


Image credit: Digital Design and Computer Architecture (2nd ed.) by Harris, H., Harris, S. (2013).

2

A computer system is a complex machine that operates on several layers of abstraction, each building upon the one below it to provide a rich set of functionalities.

At the topmost layer, we have the **Application Software**. This includes programs that perform specific tasks for users, such as word processors, web browsers, and database management systems. These applications interact with the operating system to access the hardware resources they need. EEE 111, which is the other course that you are taking, focuses on high-level programming, developing software programs using high-level language such as Python or C.

The next layer is the **Operating System**, which serves as an intermediary between the application software and the hardware. It manages resources, provides services such as file and memory management, and enforces security policies.

Beneath the operating system is the **Architecture** layer, which includes the instruction set and registers supported by the central processing unit or the CPU. This layer defines the basic capabilities of the computer, such as the operations it can perform and the data types it can handle.

The **Micro-architecture** layer is next, which includes the processor datapaths and controllers. This layer determines how the instructions are executed in the

hardware, including the organization of the processor and the design of the control unit.

Next is the **Logic** layer, which includes combinational logic such as adders, and sequential logic such as flip-flops and memories. This layer is responsible for implementing the functionality of the micro-architecture using digital logic.

The **Digital Circuits** layer follows, which includes the logic gates that make up the digital logic. These circuits are built from transistors and other electronic components.

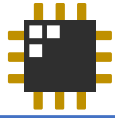
Next is the **Analog Circuits** layer, which includes the physical electronic circuits that implement the digital logic. These circuits are designed to operate on continuous signals and can include components such as resistors, capacitors, and inductors.

The **Devices** layer includes the physical devices such as transistors and diodes that make up the electronic circuits. These devices are the building blocks of the circuits and are typically fabricated on semiconductor materials.

Finally, at the bottom, we have the **Physics** layer, which includes the fundamental physical principles that underlie the operation of the electronic devices, such as the behavior of electrons in a semiconductor.

Each layer of abstraction in a computer system allows for the complexity of the lower layers to be hidden, simplifying the design and operation of the higher layers. This layered approach is a key aspect of the flexibility and power of modern computer systems.

For EEE 113, we will be briefly touching somewhere in the middle section of the abstraction layers, focusing towards the digital domain. We will touch digital circuits, logic, microarchitecture, architecture, then a little bit on the operating systems part so that we end up on a system that has a processor that can interface with input and output devices. We'll just be going over them very briefly as we only have 5 weeks to cover all of them. Looking forward, these topics will be covered in depth in EEE 143 and EEE 153, which you will be taking towards your third year of studies.



Introduction

- For this iteration of EEE 113, we will be going from the bottom up



Ready for the next adventure?

Week 5

Week 4

Week 3

Week 2

Week 1

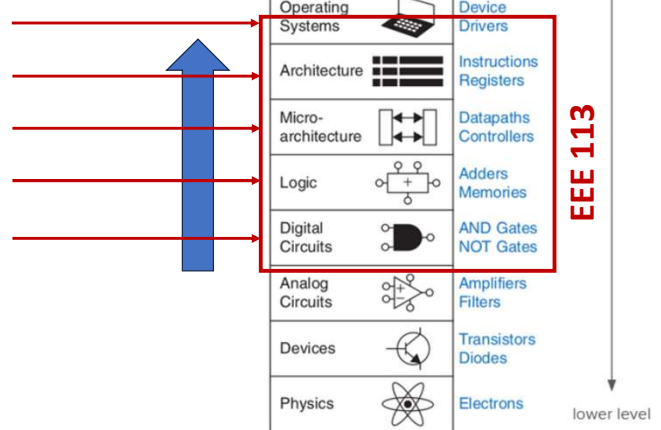
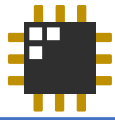


Image credit: Digital Design and Computer Architecture (2nd ed.) by Harris, H., Harris, S. (2013).

3

We will be going over the topics from the bottom up, so that you get to appreciate the smallest components first, before building up towards increasing complexity. That way, towards the end of our 5-week lectures, we will end up with something that looks complex, but you technically know that it is comprised with the small digital components that you have understood in the beginning.



Introduction

- In Computing, we usually manipulate numbers in binary (base 2).

- We only work in 0s and 1s!
- We need to be comfortable with them

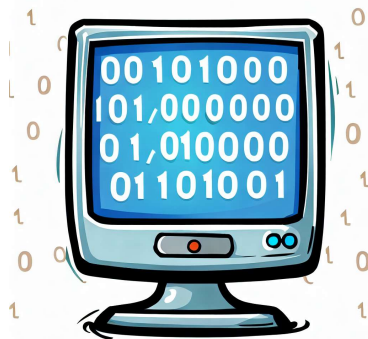


Image generated by Windows Paint Cocreator

Week 1

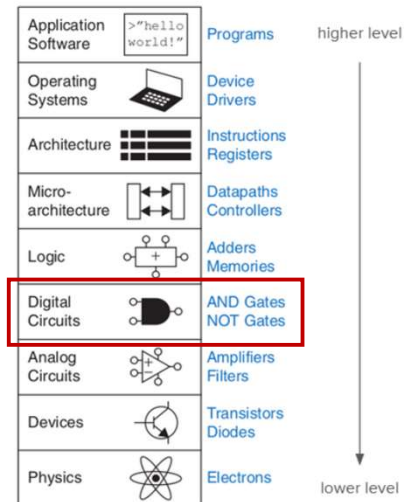
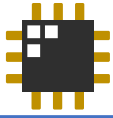


Image credit: *Digital Design and Computer Architecture* (2nd ed.) by Harris, H., Harris, S. (2013).

4

So for this week, we will start with the smallest digital component comprising a computer, logic gates.

However, before we proceed, we first have to understand some basics, especially since we are in the digital domain now. In computing, we usually manipulate numbers in binary, that is in base two. Almost everything in computing revolves around the concept of 0s and 1s. We have to be comfortable understanding what a series of 0s and 1s mean and how to interpret them.



Decimal numbers

- We represent our numbers with 10 symbols or digits

0, 1, 2, 3, 4, 5, 6, 7, 8, 9

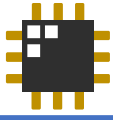
- But counting does not end in 9...
- We would need more than 1 symbol to represent larger quantities

113

$$(1 \times 10^2) + (1 \times 10^1) + (3 \times 10^0)$$

5

Decimal numbers, also known as base-10 numbers, are a common way of representing numbers in our daily life. This system uses 10 symbols or digits, ranging from 0 to 9. However, the counting does not end at 9. To represent larger numbers, we use more than one symbol or digit. For instance, the number 113 is a three-digit decimal number. It can be broken down into $1 \times 10^2 + 1 \times 10^1 + 3 \times 10^0$. Here, the digit 1 is in the hundreds place (10^2), the next digit 1 is in the tens place (10^1), and the digit 3 is in the ones place (10^0). This demonstrates how we can use multiple digits to represent larger numbers in the decimal system.



Decimal numbers

113

$$(1 \times 10^2) + (1 \times 10^1) + (3 \times 10^0)$$

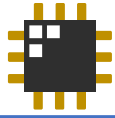
- Each digit position has different weighted values
- For decimal numbers, the digit on the left is 10 times higher than the digit on the right.

Decimal (from Latin *decimus*, meaning tenth)

6

In the decimal number system, each digit position carries a weight that is 10 times that of the position to its right. This is because the decimal system is a base-10 system. The term "decimal" comes from the Latin word "decimus", which means "tenth". This reflects the fact that each step to the left in a decimal number represents a tenfold increase in value.

For example, in the number 113, the digit 1 in the hundreds place (10^2) is 10 times the value of the digit 1 in the tens place (10^1), and 100 times the value of the digit 3 in the ones place (10^0). This pattern continues for numbers of any size, and is a fundamental characteristic of the decimal system.



Decimal numbers

- We have 10 fingers, so 10 comes out naturally!

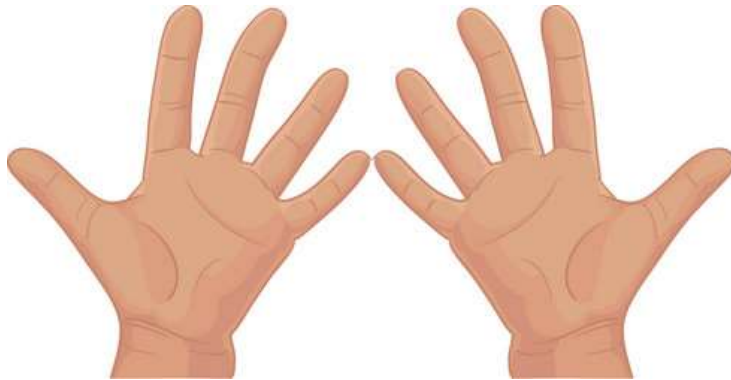
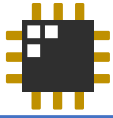


Image credit: vectorstock.com

7

The number 10 comes up very naturally because we have 10 fingers! That's how decimal numbers originally came about.



Other number systems

113

$$(1 \times 10^2) + (1 \times 10^1) + (3 \times 10^0)$$

- We have 10 digits, every digit position is a power of 10...

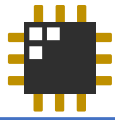
- What if we have less?



Image generated by Windows Paint Cocreator

8

So now, if we are using base 10 because we had 10 fingers, what if we had less?
We'll most likely be using a different numbering system on a different base that is not 10.



Binary numbers

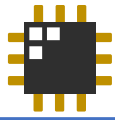
- We can represent any number in any base n , that has n digits with positional powers of n
- If we only have 2 digits, 0 and 1, how do we represent larger numbers?

$$\begin{array}{c} \text{1110}_2 \quad (\text{we say 1110 in base 2, or 1110 in binary}) \\ (1 \times 2^3) + (1 \times 2^2) + (1 \times 2^1) + (0 \times 2^0) \\ \text{2 digits (0,1) and positional powers of 2} \\ 1 \times 8 \quad + \quad 1 \times 4 \quad + \quad 1 \times 2 \quad + \quad 0 \times 1 \\ = 14_{10} \quad (\text{we say 14 in base 10, or 14 in decimal}) \end{array}$$

Indeed, just like the decimal system, we can represent any number in any base- n system, where n is the number of digits available. We can represent any number in any base n , that has n digits with positional powers of n . In the binary system, we only have two digits: 0 and 1. Therefore, we represent larger numbers using powers of 2.

For example, the binary number 1110 can be broken down as $1 \times 2^3 + 1 \times 2^2 + 1 \times 2^1 + 0 \times 2^0$. This is equivalent to $8 + 4 + 2 + 0 = 14$ in decimal.

In the binary system, each digit position carries a weight that is 2 times that of the position to its right. This is because the binary system is a base-2 system. The digit on the left is always 2 times higher than the digit on the right. This weighted positional system allows us to represent large numbers efficiently, even with just two digits. It's the fundamental principle behind digital computers and binary logic.



Binary numbers

- Base 2 is also called binary, we call the digits as *bits*

4-bit binary	Decimal value
0000	0
0001	1
0010	2
0011	3
0100	4
0101	5
0110	6
0111	7

4-bit binary	Decimal value
1000	8
1001	9
1010	10
1011	11
1100	12
1101	13
1110	14
1111	15

- If we need larger numbers, we need more bits!

10

This counting system is in base 2, also called binary. Each symbol or digit, which can be either 0 or 1, are called bits.

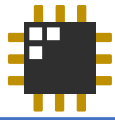
Here's a table showing the corresponding decimal values of a 4-bit binary string. We can perform the math described earlier to actually prove that each of these 4-bit binary strings are equivalent to their decimal counterparts.

When we start counting in binary, we begin with 0000, which is equivalent to 0 in decimal. Then we move to 0001, which is equivalent to 1 in decimal.

But here's where things get interesting. Since we only have two digits in binary, we've run out of digits after 1. So, to represent the next number (2 in decimal), we need to use another bit position. We reset the rightmost bit back to 0 and set the next bit to the left to 1, giving us 0010 in binary.

This process continues as we count higher. Each time we run out of digits, we add another bit position to the left and continue counting. This is how we can represent any number using just two digits in the binary system. This is analogous to the decimal numbering system, where when we start running out of digits in 1 position, we have to add 1 to the next digit position to the left. In binary, we only have 2 digits to work with, so we tend to use up the higher bit positions rather quickly. As you can see, by the time we reach decimal number 15, to represent that in binary, we already need 4 bit positions, nothing less.

Moreover, when they are laid out like this, you start to see some patterns. As you can see, the pattern in the 4-bit binary string changes as we increase the value from 0 to 15. The rightmost bit (also known as the least significant bit) toggles between 0 and 1 for every increment. The second bit from the right toggles after every 2 increments, the third bit toggles after every 4 increments, and the leftmost bit (also known as the most significant bit) toggles after every 8 increments. This pattern continues for larger binary strings as well. It's a fundamental characteristic of the binary number system.



Other bases

- Segue: How about base-1?



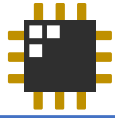
Image credit: shutterstock.com

11

A little aside and something to think about. We were focusing on binary, which is base 2, but what does it look like if we have a base 1 number system?

If you can recall from the math earlier, every digit position would be a power of the base. Thus, every bit position in base 1 number system is a power of 1. But remember, all powers of 1 are just 1 itself. Therefore, digit position has equal weights and all the weights are 1! You would need n sticks to represent the number n , for example.

Counting sticks like this, which is something you might have done as a child, is essentially using a base 1 number system.



Other bases

- Segue: How about base-64?

<https://www.youtube.com/watch?v=dQw4w9WgXcQ>

Trivia: Youtube **video IDs** are encoded in base-64!



Base 64 = 64 symbols

- a, b, c, ..., z = **26** symbols
- A, B, C, ..., Z = **26** symbols
- 0, 1, 2, ..., 9 = **10** symbols
- - and _ = **2** symbols

$$26+26+10+2 = 64$$

12

Did you know that Youtube links identify the uploaded videos in their platform using base 64?

With base 64, you need to have 64 symbols. Youtube utilizes the following to represent the symbols:

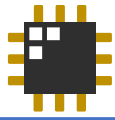
Smaller case letters a-z, which is 26 symbols.

Uppercase letters A-Z, which is also 26 symbols.

All decimal numbers 0-9, which is 10 symbols.

And the - and _ symbols.

All in all, there is $26+26+10+2 = 64$ symbols!



Binary number conversion

- This demonstrates conversion from binary to decimal
 - For every bit position **going to the left, we increase the power of 2**, then just **multiply the actual bit** to that power of 2

$$\begin{array}{c} 1110_2 \\ (1 \times 2^3) + (1 \times 2^2) + (1 \times 2^1) + (0 \times 2^0) \\ 1 \times 8 + 1 \times 4 + 1 \times 2 + 0 \times 1 \\ = 14_{10} \end{array}$$

13

Let's delve into the conversion from binary to decimal using the binary number 1110 as an example.

From the math shown earlier, which is also replicated here, for every bit position **going to the left, we increase the power of 2**, then just **multiply the actual bit** to that power of 2, add them up then we have our decimal equivalent.

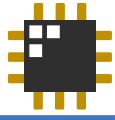
In the binary system, each digit (or bit) represents a power of 2. Starting from the rightmost bit (also known as the least significant bit), we have 2^0 , then moving to the left, we increase the power by 1 for each bit position.

So, for the binary number 1110:

- The rightmost bit is 0, and it's in the position of 2^0 . So, we calculate $0 * 2^0 = 0$.
- The next bit to the left is 1, and it's in the position of 2^1 . So, we calculate $1 * 2^1 = 2$.
- The next bit to the left is 1, and it's in the position of 2^2 . So, we calculate $1 * 2^2 = 4$.
- The leftmost bit (also known as the most significant bit) is 1, and it's in the position of 2^3 . So, we calculate $1 * 2^3 = 8$.

Finally, we add up these values: $8 + 4 + 2 + 0 = 14$. So, 1110 in binary is

equivalent to 14 in decimal. This is how we convert from binary to decimal.



Binary number conversion

- Practice, what is the decimal equivalent of this?

1110001₂

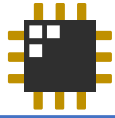
14

Let's practice, what is the decimal equivalent of this binary number?

$$1 \times 2^6 + 1 \times 2^5 + 1 \times 2^4 + 0 \times 2^3 + 0 \times 2^2 + 0 \times 2^1 + 1 \times 2^0$$

$$1 \times 64 + 1 \times 32 + 1 \times 16 + 0 \times 8 + 0 \times 4 + 0 \times 2 + 1 \times 1$$

$$64 + 32 + 16 + 1 = 113$$



Binary number conversion

- Practice, what is the decimal equivalent of this?

1110001_2

$$(\underline{1}x2^6) + (\underline{1}x2^5) + (\underline{1}x2^4) + (\underline{0}x2^3) + (\underline{0}x2^2) + (\underline{0}x2^1) + (\underline{1}x2^0)$$

$$(\underline{1}x64) + (\underline{1}x32) + (\underline{1}x16) + (\underline{0}x8) + (\underline{0}x4) + (\underline{0}x2) + (\underline{1}x1)$$

$$64 + 32 + 16 + 0 + 0 + 0 + 1$$

113_{10}

15

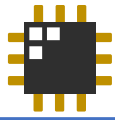
Starting from the right we have 2^0 , then as we move to the left we increase the exponent 2^1 , 2^2 , until the left most bit which ended up at 2^6 .

Then we simply multiply the bits with their corresponding weights.

$$1x2^6 + 1x2^5 + 1x2^4 + 0x2^3 + 0x2^2 + 0x2^1 + 1x2^0$$

$$1x64 + 1x32 + 1x16 + 0x8 + 0x4 + 0x2 + 1x1$$

$$64 + 32 + 16 + 1 = 113$$



Binary number conversion

- How to do the reverse? Brute-force way: Successive division

- Divide the number by the base (in this case 2). Get the quotient.
- Continuously divide the quotient by the base until it becomes 0.
- Always take note of the remainder each time the quotient is divided.
- Arrange the order of remainders. The last remainder is at the left-most position.

$$14 / 2 = 7 \text{ remainder } 0$$

$$7 / 2 = 3 \text{ remainder } 1$$

$$3 / 2 = 1 \text{ remainder } 1$$

$$1 / 2 = 0 \text{ remainder } 1$$

↑
Read in this direction

1110₂

16

Now, let's try to do the reverse process: converting a decimal number into a binary number.

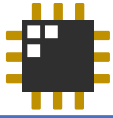
Divide the number by the base (in this case 2). Get the quotient.
Continuously divide the quotient by the base until it becomes 0.
Always take note of the remainder each time the quotient is divided.
Arrange the order of remainders. The last remainder is at the left-most position.

Let's convert the decimal number 14 to binary using successive division:

1. Divide 14 by 2, which equals 7 with a remainder of 0. So, the first digit of the binary number (from right to left) is 0.
2. Divide the quotient 7 by 2, which equals 3 with a remainder of 1. So, the next digit is 1.
3. Divide the quotient 3 by 2, which equals 1 with a remainder of 1. So, the next digit is 1.
4. Divide the quotient 1 by 2, which equals 0 with a remainder of 1. So, the next digit is 1.

Now that the quotient is 0, we stop the process. Reading the remainders from bottom to top (or from right to left), we get the binary number 1110. So, 14 in

decimal is equivalent to 1110 in binary. This is how we convert from decimal to binary using successive division.



Binary number conversion

- How to do the reverse? Faster(?) way

- Memorize the powers of 2.
- Subtract from the number the highest power of 2 that is less than that number.
- Work from the highest position (power of 2), then work towards the right continually subtracting the power of 2. Don't subtract if it becomes negative.

We are basically doing the process in reverse (starting from the highest power of 2 possible) Highest power of 2 less than 14 is $8 = 2^3$. We start from there.

$$(_x2^3) + (_x2^2) + (_x2^1) + (_x2^0) = 14$$

17

While successive division works, it can be tedious, especially for large numbers. A relatively faster way is to directly write out the binary string slowly starting from the left-most bit then working your way to the right.

This relies on you memorizing the powers of 2, or at least be familiar with it, and using subtraction. Here's how you can do it:

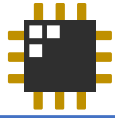
1. Memorize the powers of 2: 1, 2, 4, 8, 16, 32, 64, 128, 256, 512, 1024, and so on.

2. Start with the highest power of 2 that is less than or equal to the number you're converting. For 14, this would be 8 (2^3).

3. Subtract this power of 2 from your number. For 14, you would subtract 8 to get 6.

4. Move to the next lower power of 2 (which is 4 or 2^2 in this case) and see if you can subtract it from the remainder without getting a negative number. If you can, subtract it and put a 1 in that bit position. If you can't, put a 0 in that bit position. For 6, you can subtract 4, so you put a 1 in the 2^2 position.

5. Repeat this process with the new remainder and the next lower power of 2. Continue until you reach 2^0 .



Binary number conversion

- How to do the reverse? Faster(?) way

- Memorize the powers of 2.
- Subtract from the number the highest power of 2 that is less than that number.
- Work from the highest position (power of 2), then work towards the right continually subtracting the power of 2. Don't subtract if it becomes negative.

We are basically doing the process in reverse (starting from the highest power of 2 possible) Highest power of 2 less than 14 is $8 = 2^3$. We start from there.

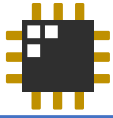
$$\begin{array}{ccccccc} (1 \times 2^3) & + & (_ \times 2^2) & + & (_ \times 2^1) & + & (_ \times 2^0) = 14 \\ 8 & + & ? & + & ? & + & ? = 14 \end{array}$$

18

So, for 14, the process would look like this:

- $14 - 8 (2^3) = 6$, so the bit at position 2^3 is 1.
- $6 - 4 (2^2) = 2$, so the bit at position 2^2 is 1.
- $2 - 2 (2^1) = 0$, so the bit at position 2^1 is 1.
- Finally, there's nothing to subtract from 0, so the bit at position 2^0 is 0.

So, 14 in decimal is 1110 in binary. This method can be faster because it eliminates the need for division and focuses on subtraction and simple binary decisions (can I subtract this power of 2 or not?). However, it does require you to have the powers of 2 memorized, at least up to the range of numbers you're working with.



Binary number conversion

- How to do the reverse? Faster(?) way

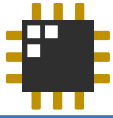
- Memorize the powers of 2.
- Subtract from the number the highest power of 2 that is less than that number.
- Work from the highest position (power of 2), then work towards the right continually subtracting the power of 2. Don't subtract if it becomes negative.

We are basically doing the process in reverse (starting from the highest power of 2 possible)
Highest power of 2 less than 14 is $8 = 2^3$. We start from there.

$$\begin{array}{ccccccc} (1 \times 2^3) & + & (1 \times 2^2) & + & (_ \times 2^1) & + & (_ \times 2^0) = 14 \\ 8 & + & 4 & + & ? & + & ? = 14 \end{array}$$

19

- $6 - 4 (2^2) = 2$, so the bit at position 2^1 is 1.



Binary number conversion

- How to do the reverse? Faster(?) way

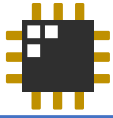
- Memorize the powers of 2.
- Subtract from the number the highest power of 2 that is less than that number.
- Work from the highest position (power of 2), then work towards the right continually subtracting the power of 2. Don't subtract if it becomes negative.

We are basically doing the process in reverse (starting from the highest power of 2 possible) Highest power of 2 less than 14 is $8 = 2^3$. We start from there.

$$\begin{array}{ccccccc} (1 \times 2^3) & + & (1 \times 2^2) & + & (1 \times 2^1) & + & (_ \times 2^0) & = & 14 \\ 8 & + & 4 & + & 2 & + & ? & = & 14 \end{array}$$

20

- $14 - 8 = 6$, so the bit at position 2^3 is 1.



Binary number conversion

- How to do the reverse? Faster(?) way

- Memorize the powers of 2.
- Subtract from the number the highest power of 2 that is less than that number.
- Work from the highest position (power of 2), then work towards the right continually subtracting the power of 2. Don't subtract if it becomes negative.

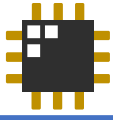
We are basically doing the process in reverse (starting from the highest power of 2 possible) Highest power of 2 less than 14 is $8 = 2^3$. We start from there.

$$\begin{array}{rcccccc} (1 \times 2^3) & + & (1 \times 2^2) & + & (1 \times 2^1) & + & (0 \times 2^0) & = & 14 \\ 8 & + & 4 & + & 2 & + & 0 & = & 14 \end{array}$$

21

- Finally, there's nothing to subtract from 0, so the bit at position 2^0 is 0.

So, 14 in decimal is 1110 in binary. This method can be faster because it eliminates the need for division and focuses on subtraction and simple binary decisions (can I subtract this power of 2 or not?). However, it does require you to have the powers of 2 memorized, at least up to the range of numbers you're working with.



Binary number conversion

- Practice, write 43 in binary. You can use either method.

43₁₀

22

Let's have another practice. Try converting 43 in binary.

Using the division method:

$$43/2 = 21 \text{ remainder } 1$$

$$21/2 = 10 \text{ remainder } 1$$

$$10/2 = 5 \text{ remainder } 0$$

$$5/2 = 2 \text{ remainder } 1$$

$$2/2 = 1 \text{ remainder } 0$$

$$1/2 = 0 \text{ remainder } 1$$

Arrange the remainder from the bottom going up. $101011 = 43$.

The process looks short, but I personally find it rather tedious. Now, let's try to use the subtraction process described earlier to do the conversion.

Largest power of 2 that can be subtracted from 43 is 32. 32 is 2^5 .

Put 1 on the 2^5 position. We start from there.

$$43 - 32 = 11 \text{ (the number we represent next)}$$

2^4 position has weight of 16, don't subtract from 11 (because $11-16$ is negative). Put 0 on the 2^4 position.

2^3 position has weight of 8, we can subtract 8 from 11. Put 1 on the 2^3 position.

$11 - 8 = 3$ (the number we represent next)

2^2 position has weight of 4, don't subtract from 3 (because $3 - 4$ is negative). Put 0 on the 2^2 position.

2^1 position has weight of 2, we can subtract 2 from 3. Put 1 on the 2^1 position.

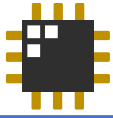
$3 - 2 = 1$ (the number we represent next)

2^0 position has weight of 1, we can subtract 1 from 1, so put 1 on the 2^0 position.

At this point, we have already reached the bit position 2^0 , so we end here.

$43 = 101011$

The process looks lengthy because I'm explaining the process along the way, but once you get the hang of it, I guarantee that it can be a faster method for conversion!



Hexadecimal numbers

- Another common base is base 16 (called hexadecimal or hex)

4-bit binary	Hex value	Decimal value
0000	0	0
0001	1	1
0010	2	2
0011	3	3
0100	4	4
0101	5	5
0110	6	6
0111	7	7

4-bit binary	Hex value	Decimal value
1000	8	8
1001	9	9
1010	A	10
1011	B	11
1100	C	12
1101	D	13
1110	E	14
1111	F	15

- In hex, there are 16 digits (0 to F).
- We usually write it with 0x prefix (e.g. 0xA, 0xF, 0xAF)
- It might be helpful to memorize this table as they come up a lot!

23

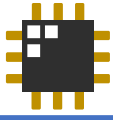
Another common base that comes up in computing is base 16, also called hexadecimal number or hex for short.

In hex, we have 16 digits, because we are in base 16. The 16 digits are represented by symbols going from 0 to 9, then letters A to F. 0 to F is actually 16 items, those will be our digits.

Here's the same table earlier for a 4-bit binary string and the equivalent decimal value. Then, we added the corresponding hex value as well. Since this table is 16 items, everything fits in a single digit of hex. You can see their equivalence here. The digits are similar to decimal from 0 to 9, but once we exceed 9, hex goes to A, then B, then C, and so on until F.

For hex, we usually write it with a 0x prefix (examples: 0xA, 0xF, 0xAF)

This set of 4-bit binary, the equivalent hex digit, and the corresponding decimal value comes up a lot in the world of computing. While, of course, you should be able to convert a number into different bases, it is very helpful if you could actually memorize this table, so that you can do your conversions faster.



Binary to hex conversion

- Practice, write 1010 1101 1111 0001 in hex

1010 1101 1110 0001₂

24

Let's have some practice. Try writing 1010 1101 1111 0001 from binary to hex.
Straight conversion from the table (or if you memorized it)

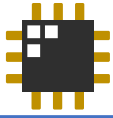
1010 = A

1101 = D

1110 = E

0001 = 1

0xADE1



Binary to hex conversion

- Practice, write 1010 1101 1111 0001 in hex

1010 1101 1110 0001₂

A D E 1

0xADE1 or ADE1₁₆

25

We can group the binary numbers into 4. You can verify from the binary to hex table shown earlier that there is always a unique mapping from 4-bit binary to hex. This unique mapping is simply because hexadecimal, or base 16, is basically 2^4 , basically implying that hex is just binary considering 4 at a time.

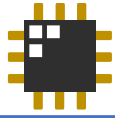
1010 is hexadecimal digit A

1101 is hexadecimal digit D

1110 is hexadecimal digit E

0001 is hexadecimal digit 1

So combining all the terms, we have 0xADE1.



Hex to decimal conversion

- Practice, what is the decimal equivalent of 0xAF?

$$0xAF = AF_{16}$$

26

Here's another one. What is the decimal equivalent of 0xAF?

There are several ways to do this:

Straight from hex to decimal. Take note how every digit position in hex is a power of 16.

$$A \times 16^1 = 10 \times 16 = 160$$

$$F \times 16^0 = 15 \times 1 = 15$$

$$160 + 15 = 175$$

Can convert to binary, then binary to decimal.

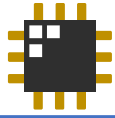
AF = 1010 1111 (using the table or if you memorized it already)

Now that we are in the binary domain, we can start doing the math from binary to decimal. Every bit position in binary is a power of 2.

$$1 \times 2^7 + 0 \times 2^6 + 1 \times 2^5 + 0 \times 2^4 + 1 \times 2^3 + 1 \times 2^2 + 1 \times 2^1 + 1 \times 2^0$$

$$1 \times 128 + 0 \times 64 + 1 \times 32 + 0 \times 16 + 1 \times 8 + 1 \times 4 + 1 \times 2 + 1 \times 1$$

$$128 + 32 + 8 + 4 + 2 + 1 = 175$$



Hex to decimal conversion

- Practice, what is the decimal equivalent of 0xAF?

$$\mathbf{0xAF = AF_{16}}$$

$$(\mathbf{\underline{A}} \times 16^1) + (\mathbf{\underline{F}} \times 16^0) = ?$$

$$(\mathbf{\underline{10}} \times 16^1) + (\mathbf{\underline{15}} \times 16^0) = ?$$

$$(\mathbf{\underline{10}} \times 16) + (\mathbf{\underline{15}} \times 1) = ?$$

$$160 + 15 = \mathbf{175_{10}}$$

27

Starting from the rightmost digit, that is 16^0 , then the one on the left is 16^1 . Then we just multiply the digits the we see with their corresponding positional power.

$$A \times 16^1 + F \times 16^0$$

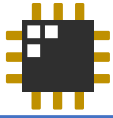
A is actually 10, while F is actually 15.

$$10 \times 16 = 160$$

$$15 \times 1 = 15$$

$$160 + 15 = 175$$

And that's the decimal equivalent of 0xAF.



Hex to decimal conversion

- Practice, what is the decimal equivalent of 0xAF?

$$0x\text{AF} = \text{AF}_{16}$$

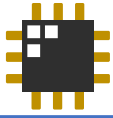
$$1010\ 1111_2$$

28

We can also convert hex into binary then convert binary to decimal. The math should work.

Converting AF to binary we get 1010 1111. You can consult the mapping table if you want to verify.

Now that we are in the binary domain, we can start doing the math from binary to decimal. Every bit position in binary is a power of 2.



Hex to decimal conversion

- Practice, what is the decimal equivalent of 0xAF?

$$0xAF = AF_{16}$$

$$1010\ 1111_2$$

$$(1 \times 2^7) + (0 \times 2^6) + (1 \times 2^5) + (0 \times 2^4) + (1 \times 2^3) + (1 \times 2^2) + (1 \times 2^1) + (1 \times 2^0)$$

$$(1 \times 128) + (0 \times 64) + (1 \times 32) + (0 \times 16) + (1 \times 8) + (1 \times 4) + (1 \times 2) + (1 \times 1)$$

$$128 + 0 + 32 + 0 + 8 + 4 + 2 + 1 = 175_{10}$$

29

Starting from the right we have 2^0 , then as we move to the left we increase the exponent 2^1 , 2^2 , until the left most bit which ended up at 2^7 .

Then we simply multiply the bits with their corresponding weights.

Then do the math and we should end up with the same answer as before. 0xAF is equal to 175 in decimal.

$$1 \times 2^7 + 0 \times 2^6 + 1 \times 2^5 + 0 \times 2^4 + 1 \times 2^3 + 1 \times 2^2 + 1 \times 2^1 + 1 \times 2^0$$

$$1 \times 128 + 0 \times 64 + 1 \times 32 + 0 \times 16 + 1 \times 8 + 1 \times 4 + 1 \times 2 + 1 \times 1$$

$$128 + 32 + 8 + 4 + 2 + 1 = 175$$